

Apache Spark Note Guide

DataFrame API, Spark SQL, Transformations & Actions

1. Getting Started in Jupyter Lab

Since you are running Spark inside a Python virtual environment with Jupyter Lab, the first step in any notebook is to initialize a `SparkSession`. This is the entry point to programming Spark with the Dataset and DataFrame API.

```
# Cell 1: Initialize Spark Session
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("SparkNotes") \
    .getOrCreate()

print(f"Spark Version: {spark.version}")
```

2. Core Concepts: Transformations vs. Actions

Spark operates on a paradigm of **Lazy Evaluation**. This means execution does not start until an action is triggered. Spark instructions are divided into two main categories:

Transformations (Lazy)

Transformations create a new DataFrame from an existing one. Because of lazy evaluation, Spark merely builds a lineage graph (Directed Acyclic Graph - DAG) of transformations. It does not execute the data processing immediately.

- `select()` : Selects specific columns.
- `filter()` or `where()` : Filters rows based on a condition.
- `withColumn()` : Adds a new column or replaces an existing one.
- `groupBy()` : Groups data for aggregation.
- `orderBy()` or `sort()` : Sorts the DataFrame.
- `join()` : Joins two DataFrames together.

Actions (Eager)

Actions trigger the execution of the DAG. They return a final value to the driver program or write data to an external storage system.

- `show()` : Prints the first n rows to the console (useful in Jupyter).
- `count()` : Returns the number of rows in the DataFrame.
- `collect()` : Returns all the records as an array of Row objects to the driver (use with caution on large datasets).
- `write()` : Saves the DataFrame to a file (CSV, Parquet, JSON, etc.).
- `first()` / `take(n)` : Returns the first row or first n rows.

3. The DataFrame API

A DataFrame is a distributed collection of data organized into named columns. It is conceptually equivalent to a table in a relational database or a data frame in R/Python (pandas), but with richer optimizations under the hood.

Tip: Unlike pandas DataFrames, Spark DataFrames are distributed across a cluster, meaning operations are processed in parallel.

```
# Cell 2: Creating a DataFrame and applying Transformations & Actions
data = [
    ("Alice", "Engineering", 75000),
    ("Bob", "Marketing", 60000),
    ("Charlie", "Engineering", 80000),
    ("David", "Sales", 65000)
]
columns = ["Name", "Department", "Salary"]

# Create DataFrame
df = spark.createDataFrame(data, columns)

# --- TRANSFORMATION ---
# Increase salary by 10% for Engineering department
from pyspark.sql.functions import col

df_transformed = df.filter(col("Department") == "Engineering") \
    .withColumn("New_Salary", col("Salary") * 1.10)

# --- ACTION ---
# This actually executes the plan
df_transformed.show()
```

4. Spark SQL

Spark SQL allows you to execute SQL queries directly on DataFrames. To do this, you must register the DataFrame as a temporary view.

```
# Cell 3: Using Spark SQL
# Register the DataFrame as a SQL temporary view
df.createOrReplaceTempView("employees")

# Execute a SQL query
sql_df = spark.sql('''
    SELECT Department, AVG(Salary) as Avg_Salary
    FROM employees
    GROUP BY Department
    ORDER BY Avg_Salary DESC
''')

# Action
sql_df.show()
```

5. Using Spark with Jupyter in Docker

Running Spark natively on your machine is great, but Docker provides a perfectly clean, isolated, and reproducible environment. The Jupyter Foundation provides pre-built Docker images for PySpark.

Step 1: Install Docker

Ensure Docker Desktop (or Docker Engine) is installed and running on your machine.

Step 2: Pull and Run the PySpark Notebook Image

Open your terminal and execute the following command:

```
docker run -it --rm -p 8888:8888 -p 4040:4040 -v "${PWD}:/home/jovyan/work" jupyter/pysp
```

- `-it --rm`: Runs interactively and removes the container when stopped.
- `-p 8888:8888`: Maps the Jupyter notebook UI port.
- `-p 4040:4040`: Maps the Spark Web UI port (useful for viewing DAGs and job performance).

- `-v "${PWD}:/home/jovyan/work"`: Mounts your current local directory to the container so your notebooks are saved on your host machine.
- `jupyter/pyspark-notebook`: The official image containing Spark, Python, and Jupyter.

Step 3: Access Jupyter

Look at the terminal output. You will see a URL that looks like:

`http://127.0.0.1:8888/lab?token=<your_token_here>`

Copy and paste that URL into your browser to start coding in Jupyter Lab with PySpark fully configured!